

Penetration Testing Report: TryHackMe - Madness

Target IP: 10.112.136.71

Platform: TryHackMe

Severity Level: High / Critical (Full System Compromise)

1. Executive Summary

During the security assessment of the target machine 10.112.136.71 (**Madness**), multiple security flaws were identified across web application controls and local system binaries. Initial access was obtained by identifying hidden web endpoints, fixing corrupt file headers to extract steganographic payloads, and brute-forcing hidden HTTP parameters. Full root-level compromise was subsequently achieved by exploiting a known Local Privilege Escalation (LPE) vulnerability in an outdated **GNU Screen** binary running with SUID privileges.

2. Reconnaissance & Enumeration

2.1 Port Scanning (Nmap)

A full TCP port scan was conducted against the target IP address to identify accessible services and host configurations.

Bash
nmap -sC -sV -oA nmap/initial 10.112.136.71

Scan Results:

Port	State	Service	Version / Details
22/tcp	Open	SSH	OpenSSH 7.2p2 Ubuntu 4ubuntu2.8
80/tcp	Open	HTTP	Apache httpd 2.4.18 ((Ubuntu))

- - **Host OS:** Linux (Ubuntu)
 - **Web Server:** Apache httpd 2.4.18 displaying the default Apache2 Ubuntu page.

2.2 Directory & Content Fuzzing (Gobuster)

To uncover non-indexed directories and hidden assets, directory brute-forcing was initiated on the HTTP service using `gobuster`.

Bash

```
gobuster dir -u http://10.112.136.71/ -w /usr/share/wordlists/dirb/common.txt
```

Output Summary:

- `/index.php` (Status: 200)
- `/.hta`, `/.htaccess`, `/.htpasswd`, `/server-status` (Status: 403)

3. Initial Access & Exploitation

3.1 Steganography Analysis (`thm.jpg`)

Inspection of the HTML source code on the default index page revealed a reference to an embedded image file `thm.jpg`.

1. **Asset Retrieval:**
2. Bash

```
wget http://10.112.136.71/thm.jpg
```

3.

4. **File Inspection & Header Repair:**

Initial analysis using `file thm.jpg` indicated the file was corrupted or improperly identified. Opening the binary in a hex editor (`hexeditor` / `xxd`) revealed missing or malformed JPEG magic bytes. Replacing the corrupted initial bytes with standard JPEG header signatures (`FF D8 FF E0`) repaired the file, rendering it viewable.

5. **Steganographic Extraction:**

Running `steghide` on the repaired image (without a passphrase) successfully extracted hidden plaintext content:

6. Bash

```
steghide extract -sf thm.jpg
```

7.

8. **Extracted Data:** Discovered a hidden endpoint path:
`/th1s_1s_h1dd3n/`

3.2 Parameter Fuzzing & SSH Credential Recovery

Navigating to `http://10.112.136.71/th1s_1s_h1dd3n/` presented a prompt requesting a secret parameter between 1 and 100.

1. Burp Suite Intruder Fuzzing:

An automated payload attack was configured against the target GET query parameter `?secret=$1-100$`.

- **Target URL:**
`[http://10.112.136.71/th1s_1s_h1dd3n/?secret=73](http://10.112.136.71/th1s_1s_h1dd3n/?secret=73)`
- **Valid Payload:** `73` (returned a unique response length and confirmed password hash/hint).

2. Hidden Passphrase Extraction:

Submitting `?secret=73` returned a secret passphrase. Cross-referencing this passphrase with the steg-extracted content and hidden HTML elements from the home page revealed the following operational SSH credentials:

- **Username:** `joker`
- **Password:** `*axA&GF8dP`

3. SSH Access & Flag 1 Recovery:

Using the recovered credentials, remote access was established via SSH:

4. Bash

```
ssh joker@10.112.136.71
```

5.

- **User Flag (`user.txt`):** Successfully read from `/home/joker/user.txt`.

4. Privilege Escalation

4.1 System Enumeration

After establishing a local user session as `joker`, automated and manual local enumeration were performed to locate local privilege escalation vectors. Checking SUID binaries revealed an outdated binary version:

Bash

```
find / -perm -4000 -type f 2>/dev/null
```

Key Discovery:

- Binary Path: `/bin/screen` or local executable path for **GNU Screen 4.5.0** (or `screen-4.00.00`).

4.2 Exploitation (CVE-2017-5618 / Local SUID Arbitrary File Write)

The installed version of `GNU Screen` is vulnerable to a local privilege escalation flaw that allows arbitrary file creation/overwriting due to improper log file path handling when running with root SUID permissions.

1. Exploitation Methodology:

- A dynamic library (**libhax.so**) was compiled locally to execute a shell payload as root.
- A root shell launcher binary (**rootshell**) was compiled to set the real and effective UID/GID to 0 (**setuid(0) / setgid(0)**).
- The SUID **screen** binary was executed using log creation arguments to append **/etc/ld.so.preload** pointing to **/tmp/libhax.so**.

2. Execution Steps:

3. Bash

Transfer exploit payload / shell scripts to target /tmp directory

cd /tmp

chmod +x exploit.sh

./exploit.sh

4.

5. Root Shell Verification & Flag 2 Recovery:

Upon running the privilege escalation script, root access was successfully spawned:

6. Bash

id

Output: uid=0(root) gid=0(root) groups=0(root)

cat /root/root.txt

7.

- **Root Flag (**root.txt**)**: Successfully retrieved from **/root/root.txt**.

5. Remediation & Recommendations

1. Update and Patch Systems:

- Upgrade **GNU Screen** to the latest version immediately to patch the SUID privilege escalation vulnerability.
- Audit all SUID binaries (**find / -perm -4000**) and remove the SUID bit from applications that do not strictly require it.

2. Web Application Hardening:

- Do not rely on "security through obscurity" (e.g., hidden parameters like **?secret=73** or obfuscated URL paths like **/th1s_1s_h1dd3n/**).
- Remove sensitive operational comments, hints, and structural files from web root directories.

3. SSH & Credential Security:

- Enforce key-based authentication for SSH access and disable password authentication entirely in **/etc/ssh/sshd_config**.